Prompts y respuestas IA

Álvaro Arcís Reverte y Alejandro Nicolás Navarro

Máster Universitario en Inteligencia Artificial
Curso 2026–2027

Prompt

Prepara un archivo README inicial, teniendo en cuenta lo pedido.

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
# Predicción Multiclase de Riesgo Cardiovascular en Población Adulta
> **Máster Universitario en Inteligencia Artificial | Universidad de
Murcia (UMU)**
> **Asignatura:** Machine Learning | **Curso:** 2026/2027
> **Colaboración clínica:** Hospital Morales Meseguer

---

## [U+FFFD][U+FFFD] Descripción del Proyecto

Las enfermedades cardiovasculares representan una de las principales
causas de morbimortalidad a nivel global. Este proyecto tiene como
objetivo diseñar, entrenar y evaluar modelos predictivos de Machine
Learning capaces de clasificar con alta precisión y fiabilidad clínica el
nivel de riesgo cardiovascular en una cohorte adulta de **2.254
pacientes**.

La variable objetivo (`riesgo_cardiovascular`) contempla tres niveles de
estratificación:
* **Clase 0:** Bajo Riesgo
* **Clase 1:** Riesgo Moderado
* **Clase 2:** Alto Riesgo

El repositorio recoge el flujo de trabajo end-to-end: desde el análisis
exploratorio de datos (EDA) y la mitigación de problemáticas de calidad
del dato (imputación, outliers, desbalanceo, escalado), hasta el ajuste
de hiperparámetros, validación cruzada y comparación estadística de
clasificadores base y ensambles.

---

## [U+FFFD][U+FFFD] Integrantes del Equipo

| Nombre y Apellidos | Usuario de GitHub | Correo Institucional |
| :--- | :--- | :--- |
| *[Nombre Miembro 1]* | `@usuario1` | `usuario1@um.es` |
| *[Nombre Miembro 2]* | `@usuario2` | `usuario2@um.es` |
| *[Nombre Miembro 3]* | `@usuario3` | `usuario3@um.es` |

---

## [U+FFFD][U+FFFD] Estructura del Repositorio

El proyecto se organiza bajo una estructura modular orientada a la
reproducibilidad científica:
```

```text
[U+251C][U+2500][U+2500] data/
[U+2502]   [U+251C][U+2500][U+2500] raw/                        # Dataset
original (enfermedad_cardiovascular.csv)
[U+2502]   [U+2514][U+2500][U+2500] processed/                  # Datasets
limpios o transformados (si aplica)
[U+251C][U+2500][U+2500] docs/                              # Documentación
técnica y entregables
[U+2502]   [U+251C][U+2500][U+2500] memoria_tecnica.pdf         # Memoria
técnica maquetada (máx. 15 páginas)
[U+2502]   [U+251C][U+2500][U+2500] uso_etico_ia.pdf            # Registro
de prompts y reflexión crítica de IA
[U+2502]   [U+2514][U+2500][U+2500] video_link.txt              # Enlace
al vídeo explicativo (máx. 7 min)
[U+251C][U+2500][U+2500] notebooks/                         # Jupyter
Notebooks de desarrollo y análisis
[U+2502]   [U+251C][U+2500][U+2500] 01_eda_calidad_datos.ipynb # Análisis
exploratorio y detección de problemáticas
[U+2502]   [U+251C][U+2500][U+2500] 02_preprocesamiento.ipynb  #
Transformaciones, codificación e imputación
[U+2502]   [U+251C][U+2500][U+2500] 03_modelado_evaluacion.ipynb#
Entrenamiento, ajuste de hiperparámetros y métricas
[U+2502]   [U+2514][U+2500][U+2500] 04_test_estadisticos.ipynb #
Comparación estadística de modelos (opcional)
[U+251C][U+2500][U+2500] src/                               # Código fuente
refactorizado y reutilizable
[U+2502]   [U+251C][U+2500][U+2500] __init__.py
[U+2502]   [U+251C][U+2500][U+2500] data_loader.py              # Carga y
comprobación de integridad
[U+2502]   [U+251C][U+2500][U+2500] preprocessing.py           # Pipelines
de preprocesamiento (Scikit-Learn)
[U+2502]   [U+251C][U+2500][U+2500] models.py                   #
Definición y pipelines de algoritmos
[U+2502]   [U+2514][U+2500][U+2500] evaluation.py               # Cálculo
de métricas y matrices de confusión
[U+251C][U+2500][U+2500] .gitignore                         # Filtro de
ficheros innecesarios
[U+251C][U+2500][U+2500] environment.yml                    # Configuración
del entorno Conda (alternativa)
[U+251C][U+2500][U+2500] requirements.txt                   # Fichero de
dependencias fijadas (versiones exactas)
[U+2514][U+2500][U+2500] README.md                          # Documento
principal del repositorio
```

Análisis

README inicial preparado, incluyendo descripción del proyecto, integrantes, estructura de carpetas y archivos. Se rellena con los datos de los miembros y la estructura del proyecto se cambiará.

Prompt

```
lee el proyecto y recomiendame una estructura para los archivos, crea los
archivos y las carpetas necesarias, pero ni se te ocurra resolverlos, e
incluye esta interacción al Uso_IA
```

Respuesta IA Claude Code (Claude Opus 5.5), CLI en Windows

```
Tras leer el enunciado, las instrucciones de entrega, el dataset y el
README, propuso y creó la siguiente estructura (solo esqueletos vacíos,
sin resolver ningún apartado):

- data/raw/            -> se movió aquí enfermedad_cardiovascular.csv
(git mv)
- data/processed/      -> vacío (.gitkeep)
- notebooks/           -> 01_eda, 02_preprocesamiento, 03_modelado,
04_evaluacion, 05_test_estadisticos (.ipynb), cada uno solo con títulos
de sección y celdas "# TODO"
- src/                 -> __init__, config, data_loader, preprocessing,
models, evaluation, statistical_tests (.py), solo con un docstring que
describe su responsabilidad y "TODO"
- results/figures/, results/tables/, models/ -> vacíos (.gitkeep)
- docs/decisiones.md   -> tabla para registrar decisiones de diseño y
avances
- docs/memoria/, docs/uso_etico_ia/ -> vacíos (.gitkeep)
- docs/video.md        -> plantilla para el enlace del vídeo
- requirements.txt     -> vacío, con instrucciones para rellenarlo con
pip freeze
- .gitignore           -> entornos virtuales, cachés, checkpoints,
modelos y auxiliares de LaTeX
- README.md            -> se rellenó la sección "Estructura del
Repositorio" y se añadió una sección "Reproducción" pendiente
```

Análisis

Se prepara la estructura de carpetas y archivos del proyecto, incluyendo los esqueletos vacíos para cada sección para facilitar la organización y el desarrollo posterior.

Conversación Alejandro para el EDA: **Ver conversación completa en GitHub**

Análisis

En el enlace se sitúa la conversación mantenida con el LLM para la resolución de problemáticas de calidad del dato, incluyendo prompts, respuestas resumidas y análisis crítico.

Prompt

en visual studio code (.ipynb) cuando muestro, por ejemplo, un dataframe, se cortan las columnas o las filas a mitad y lo mismo ocurre con los print largos. Cómo puedo ver la salida completa?

Respuesta IA Gemini Flash 3.8 en Google AI Studio

Este problema ocurre por dos razones: las limitaciones propias de Pandas (para
no colapsar la memoria) y las limitaciones de la interfaz de VS Code (para no
ralentizar el editor).

Aquí tienes las soluciones para ambos casos:

1. Para DataFrames de Pandas (Filas y Columnas cortadas)

Pandas viene configurado por defecto para truncar tablas grandes. Puedes cambiar
esta configuración ejecutando esto en una celda al principio de tu notebook:

```
import pandas as pd

# Mostrar todas las columnas
pd.set_option('display.max_columns', None)

# Mostrar todas las filas (¡Cuidado si tienes millones de filas!)
pd.set_option('display.max_rows', None)

# Mostrar el contenido completo de cada celda (que no corte el texto largo)
pd.set_option('display.max_colwidth', None)
```

Consejo Pro para VS Code:
VS Code tiene un visor de datos integrado excelente.

1. Haz clic en el botón "Variables" en la barra superior del notebook (al lado
   de "Clear All Outputs").
2. Busca tu DataFrame y haz clic en el icono pequeño que parece una tabla a la
   izquierda del nombre de la variable.
   Se abrirá una pestaña interactiva donde puedes ordenar, filtrar y ver todos
   los datos sin recortar.

2. Para salidas de texto largas (print())

Si haces un print() muy largo, VS Code a veces corta el texto o le pone una
barra de desplazamiento molesta.

Opción A: Desactivar el scroll (Hacer que se vea todo hacia abajo)

```
1.  Pasa el cursor por el margen izquierdo de la celda de salida (al lado
de
    donde aparece el texto).
2.  Haz clic en los tres puntos (...) que aparecen o haz clic derecho
sobre la
    salida.
3.  Selecciona "Toggle Output Scrolling" (Alternar desplazamiento de
salida).
    Esto expandirá la celda para que muestre todo el texto en lugar de
    recortarlo en una ventanita pequeña.

Opción B: Cambiar el límite de líneas en la configuración de VS Code

Por seguridad, VS Code limita el texto plano a unas 100 líneas por celda
para no
trabarse. Puedes aumentarlo:

1.  Ve a Configuración: Ctrl + , (en Windows/Linux) o Cmd + , (en Mac).
2.  En la barra de búsqueda escribe: notebook text line limit.
3.  Busca la opción: Notebook > Output: Text Line Limit.
4.  Aumenta el número (por defecto suele ser 100, puedes poner 500, 1000
o más).

3. Solución rápida en código para un print puntual

Si solo quieres imprimir un texto muy largo o un DataFrame como texto
completo
una sola vez sin cambiar la configuración global:

# Para DataFrames:
print(df.to_string())

# Para textos gigantes (evitar que la UI se congele guardándolo en un
archivo temporal):
with open("salida.txt", "w", encoding="utf-8") as f:
    f.write(str(tu_variable_gigante))
# Y luego lo abres directamente en VS Code con Ctrl+O
```

Análisis

Se acepta el código para poder visualizar los dataframes que pandas corta por defecto, dejándolo como código para descomentar si se necesita.